



FULLSTACK 3

Evaluación 3



21 DE JUNIO DE 2026
SCARLET JATA – VICENTE AVILA – PAOLA NARR
Ducos LLC

Contenido

1. Introducción y contexto	1
2. Arquitectura final	2
2.1 Diagrama de Contexto	2
2.2 Arquitectura interna	3
2.3 Componentes	4
2.4 Diagrama de contenedores	5
2.5 Flujo de tráfico, arranque y despliegue	6
3. Patrones de diseño y arquitectónicos aplicados.....	8
3.1 Patrones arquitectónicos	8
3.2 Patrones de diseño	9
4. Modelo de datos	10
4.1 Agregados por servicio.....	10
4.2 Enlaces lógicos entre dominios (sin FK cruzada).....	12
4.3 Máquinas de estado.....	12
4.4 Convención de nomenclatura	12
5. Estrategia e implementación de pruebas	12
5.1 Enfoque	12
5.2.1 Evidencia de ejecución	13
5.2 Resumen de pruebas por microservicio	14
6. Calidad y cobertura: análisis con SonarQube.....	14
6.1 Resultados.....	15
6.2 Cómo reproducir el análisis.....	16
7. Integración Continua (CI/CD).....	17
7.1 Jobs del pipeline	17
7.2 Estado.....	17
8. Refactorización y mejoras	17
9. Conclusiones.....	18

1. Introducción y contexto

SmartLogix es una plataforma logística para PYMEs de eCommerce, desarrollada a lo largo del semestre en tres etapas: diseño de arquitectura (EV1), construcción de los componentes frontend y backend (EV2) e integración, pruebas y aseguramiento de calidad.

El objeti

Antes de presentar la arquitectura interna de SmartLogix, se muestra el siguiente diagrama de contexto, el cual permite visualizar los límites del sistema y sus principales interacciones con actores y servicios externos.

vo de esta etapa final es consolidar la solución en un sistema funcional e integrado, garantizando que el código cumple estándares de calidad mediante:

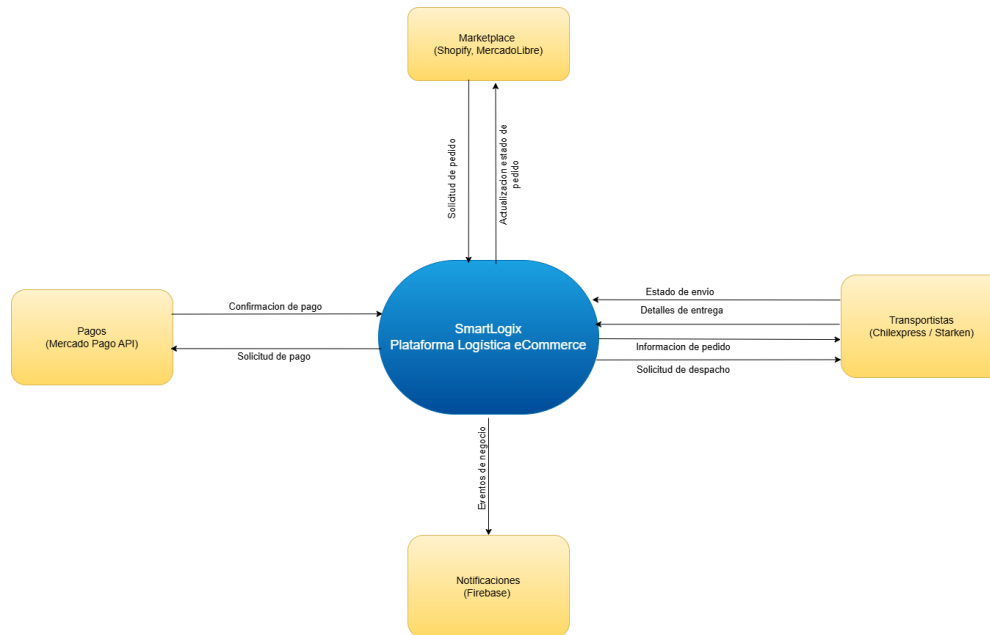
- **Pruebas unitarias** con cobertura ≥ 60 %, validada con **SonarQube**.
- **Refactorización** y mejora del código.
- **Integración Continua (CI)** que ejecuta las pruebas automáticamente en cada cambio.
- **Documentación final**: arquitectura final, modelo de datos y estrategia de pruebas.

Este informe describe el estado final del sistema. Donde la nomenclatura difiere de etapas previas (los microservicios y la API pública migraron a inglés), se usa la versión vigente del código.

2. Arquitectura final

Antes de presentar la arquitectura interna de SmartLogix, se muestra el siguiente diagrama de contexto, el cual permite visualizar los límites del sistema y sus principales interacciones con actores y servicios externos.

2.1 Diagrama de Contexto



SmartLogix actúa como plataforma central para la gestión logística de PYMEs de comercio electrónico. Los usuarios interactúan con la solución mediante una aplicación web, mientras que servicios externos especializados proporcionan capacidades complementarias como autenticación, notificaciones, pagos y coordinación de entregas. Esta integración permite mantener desacopladas las responsabilidades de cada componente y facilita la evolución independiente de la plataforma.

Nota: Las integraciones externas mostradas en el diagrama se incluyen como referencia arquitectónica y no forman parte de las funcionalidades implementadas en esta versión del sistema.

2.2 Arquitectura interna

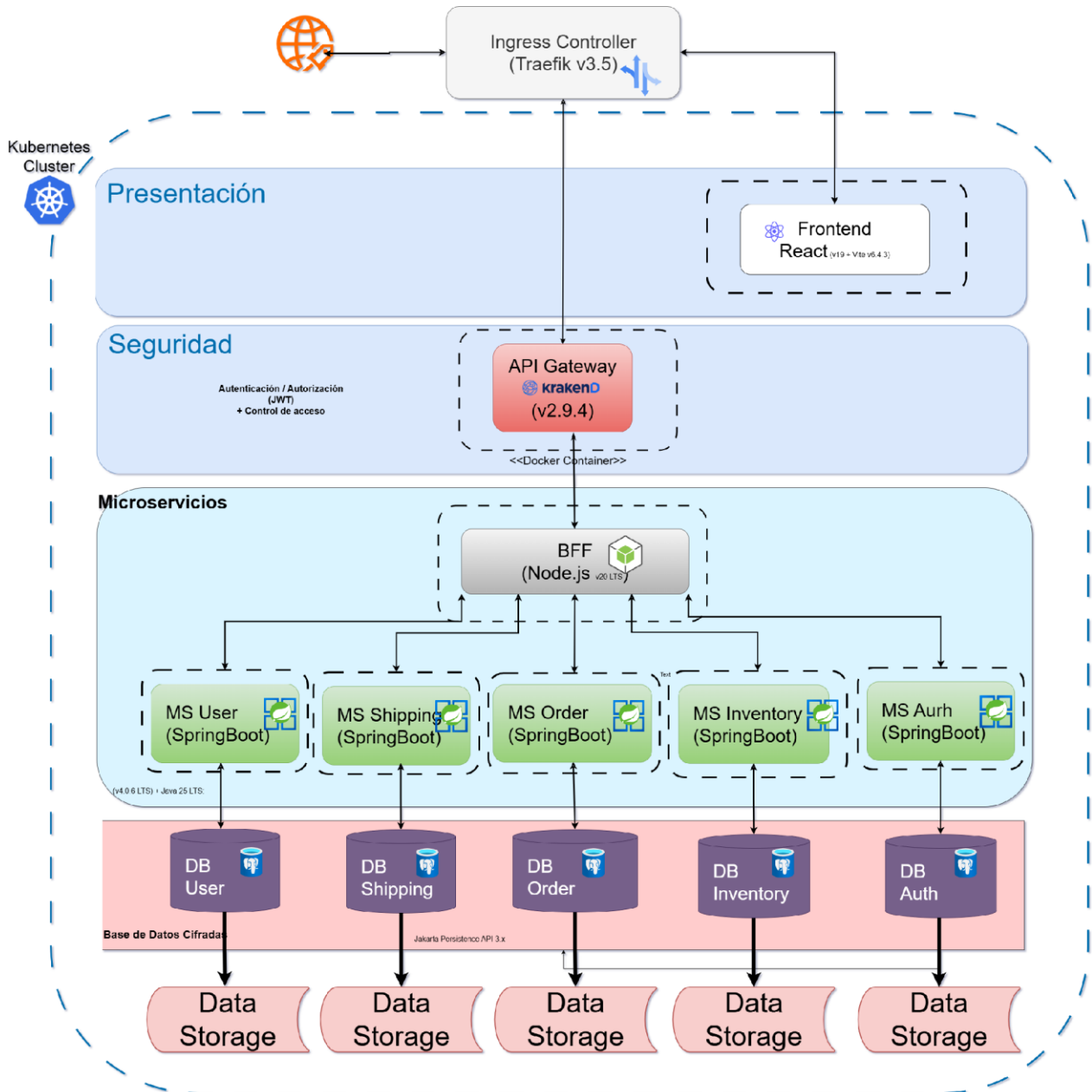
La solución es una arquitectura de microservicios con dos capas de borde (Ingress + API Gateway), un Backend For Frontend (BFF) que orquesta y una SPA.

Cada microservicio es dueño de su propia base de datos (Database per Service).

2.3 Componentes

Componente	Rol	Tecnología	Versión	Puerto
Traefik (Ingress)	Routing por host, TLS, middlewares de seguridad	Traefik	3.5	80 / 443
Frontend (SPA)	Interfaz del operador (Login, Dashboard, envíos, bodegas, AI Hub)	React + Vite + TypeScript + Tailwind	React 19 / Vite 6 / TS 5.8	—
KrakenD (API Gateway)	Routing /api/, rate-limit, validación JWT RS256, CORS	KrakenD	2.9.4	8080
BFF	Orquestación, composición de respuestas, saga de checkout, proxy CRUD	Node.js + Express + zod	Node 20 / Express 4	3000
ms-auth	Login/registro, emisión de JWT RS256, JWKS	Spring Boot + Java	3.5.0 / Java 25	8081
ms-user	Directorio de usuarios con perfil extendido	Spring Boot + Java	4.0.6 / Java 25	8080
ms-order	Pedidos + máquina de estados + auditoría	Spring Boot + Java	4.0.6 / Java 25	8080
ms-inventory	Productos, bodegas, stock (optimistic locking), movimientos	Spring Boot + Java	4.0.6 / Java 25	8080
ms-shipping	Envíos, transportistas, seguimiento + máquina de estados	Spring Boot + Java	4.0.6 / Java 25	8080
PostgreSQL x5	Una base aislada por servicio (DB-per-Service)	PostgreSQL	16-alpine	—

2.4 Diagrama de contenedores



2.5 Flujo de tráfico, arranque y despliegue

El tráfico entra por Traefik, que separa el frontend estático (/) del tráfico de API (/api/). KrakenD valida el JWT (firma RS256 contra el JWKS de ms-auth) y el scope requerido antes de reenviar al BFF, que orquesta los microservicios.

El arranque se encadena con health probes (Actuator) y `depends\_on: service\_healthy: db- → ms- → bff → api-gateway`.

El sistema se despliega en dos modos equivalentes:

- **Docker Compose** (desarrollo local): ingress Traefik con middlewares declarativos (secure-headers, rate-limit, cors-api).

Containers [Give feedback](#)

Container CPU usage ⓘ
1109.94% / 1400% (14 CPUs available)

Container memory usage ⓘ
4.7GB / 22.66GB

☒ Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk res	Actions
☐	▼ ⓘ	smartlogix	-	-	-	967%	1.58GB / 301.6GI	6.78%	241.11M	
☐	●	api-gateway	37629f97a5ac	smartlogix	8090:8080 ↗	0%	17.51MB / 23.2G	0.07%	63.7MB	
☐	●	bff	4e35d964cc48	smartlogix		0.01%	69.88MB / 23.2G	0.29%	68.8MB	
☐	●	ms-auth	046fc361bc51	smartlogix		0.09%	471.2MB / 23.2G	1.98%	56MB /	
☐	●	frontend	781313b9c6fa	smartlogix	5173:80 ↗	0%	12.72MB / 23.2G	0.05%	7.74MB	
☐	●	ms-shipping	c3c8cb770993	smartlogix		270.96%	275.9MB / 23.2G	1.16%	0B / 45.	
☐	●	ms-inventory	8aecdb9a5a5	smartlogix		124.87%	278.1MB / 23.2G	1.17%	0B / 94.	
☐	●	ms-order	c57079b97d72	smartlogix		222.22%	47.26MB / 23.2G	0.2%	0B / 0B	
☐	●	ms-user	2e0361f3bc4c	smartlogix		335.63%	278.3MB / 23.2G	1.17%	0B / 411	
☐	●	db-user	fe71a60c63af	postgres:1		0%	22.62MB / 23.2G	0.1%	4.67MB	
☐	●	db-inventory	4ac3b9b4a050	postgres:1		13.22%	36.9MB / 23.2GB	0.16%	3.28MB	

- **Kubernetes** (kustomize): el mismo ingress Traefik se reproduce con CRDs nativos (IngressRoute + Middleware), conservando idénticas políticas de borde.

Kubernetes [Give feedback](#)

Namespace
smartlogix

Cluster Stop Edit cluster

Cluster	Cluster type	Nodes	Kubernetes version
Active	kind	1	v1.34.3

Deployments

Name	Status	Pods
frontend	Available	1/1
ms-auth	Available	1/1
ms-inventory	Available	1/1
ms-order	Available	1/1
ms-shipping	Available	1/1

Pods

Name	Status
api-gateway-7c95d89dcf-jhjpr	Running
bff-57b8f8c478-s9dcw	Running
db-auth-0	Running
db-inventory-0	Running
db-order-0	Running

Kubernetes [Give feedback](#)

ms-shipping	Available	1/1
-------------	-----------	-----

db-order-0	Running
------------	---------

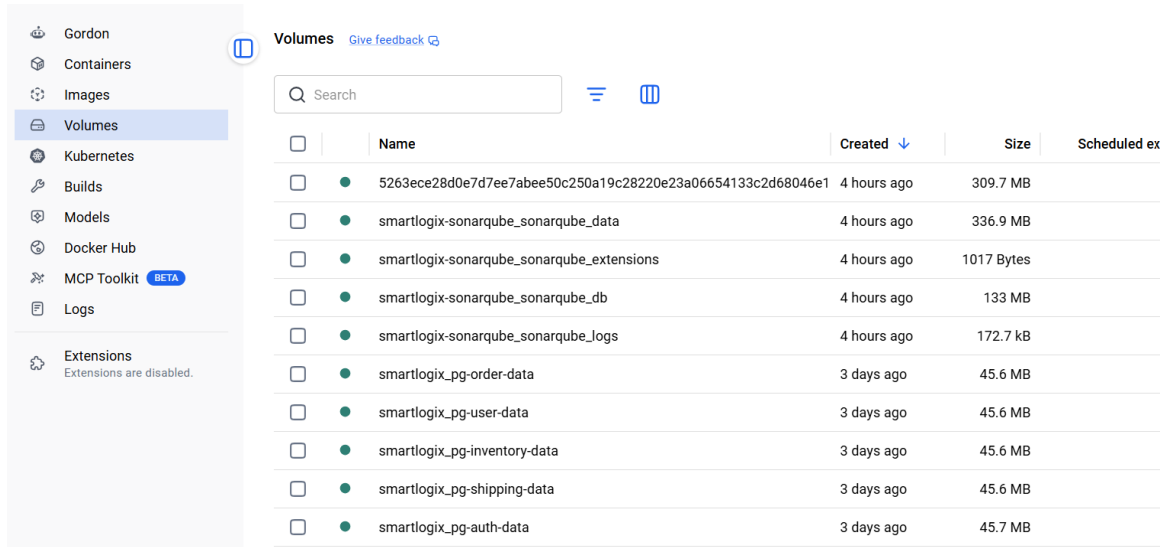
Nodes

Name	Status
desktop-control-plane	Ready

Services

Name	Cluster IP	Ports
api-gateway	10.96.23.74	8080/TCP
bff	10.96.221.207	3000/TCP
db-auth	None	5432/TCP
db-inventory	None	5432/TCP
db-order	None	5432/TCP

Cada microservicio incluye su Deployment, Service y su PostgreSQL (StatefulSet con volumen persistente). Los servicios stateless escalan horizontalmente vía réplicas/HPA, balanceados por el Service de Kubernetes.



☐	Name	Created ↓	Size	Scheduled ex
☐	5263ece28d0e7d7ee7abee50c250a19c28220e23a06654133c2d68046e1	4 hours ago	309.7 MB	
☐	smartlogix-sonarqube_sonarqube_data	4 hours ago	336.9 MB	
☐	smartlogix-sonarqube_sonarqube_extensions	4 hours ago	1017 Bytes	
☐	smartlogix-sonarqube_sonarqube_db	4 hours ago	133 MB	
☐	smartlogix-sonarqube_sonarqube_logs	4 hours ago	172.7 kB	
☐	smartlogix_pg-order-data	3 days ago	45.6 MB	
☐	smartlogix_pg-user-data	3 days ago	45.6 MB	
☐	smartlogix_pg-inventory-data	3 days ago	45.6 MB	
☐	smartlogix_pg-shipping-data	3 days ago	45.6 MB	
☐	smartlogix_pg-auth-data	3 days ago	45.7 MB	

3. Patrones de diseño y arquitectónicos aplicados

3.1 Patrones arquitectónicos

- **Microservicios + Database per Service** — dominios de negocio (pedido, inventario, envío) más identidad (auth) y usuarios, cada uno con su PostgreSQL. No hay claves foráneas cruzadas: los IDs entre dominios son identificadores lógicos que cada servicio interpreta vía su contrato REST.
- **API Gateway + Ingress separados** — Traefik resuelve lo cross-cutting (TLS, ingress, frontend); KrakenD se especializa en /api/ con políticas declarativas (rate-limit, validación JWT, CORS).
- **Backend For Frontend (BFF)** — capa Node.js que compone respuestas de varios MS, orquesta flujos transaccionales y hace passthrough del CRUD simple. Schema-first migrations (Flyway) — cada MS versiona su schema; Hibernate corre en ddl-auto=validate.
- **Health Probes (Actuator)** — /actuator/health para arranque ordenado.

3.2 Patrones de diseño

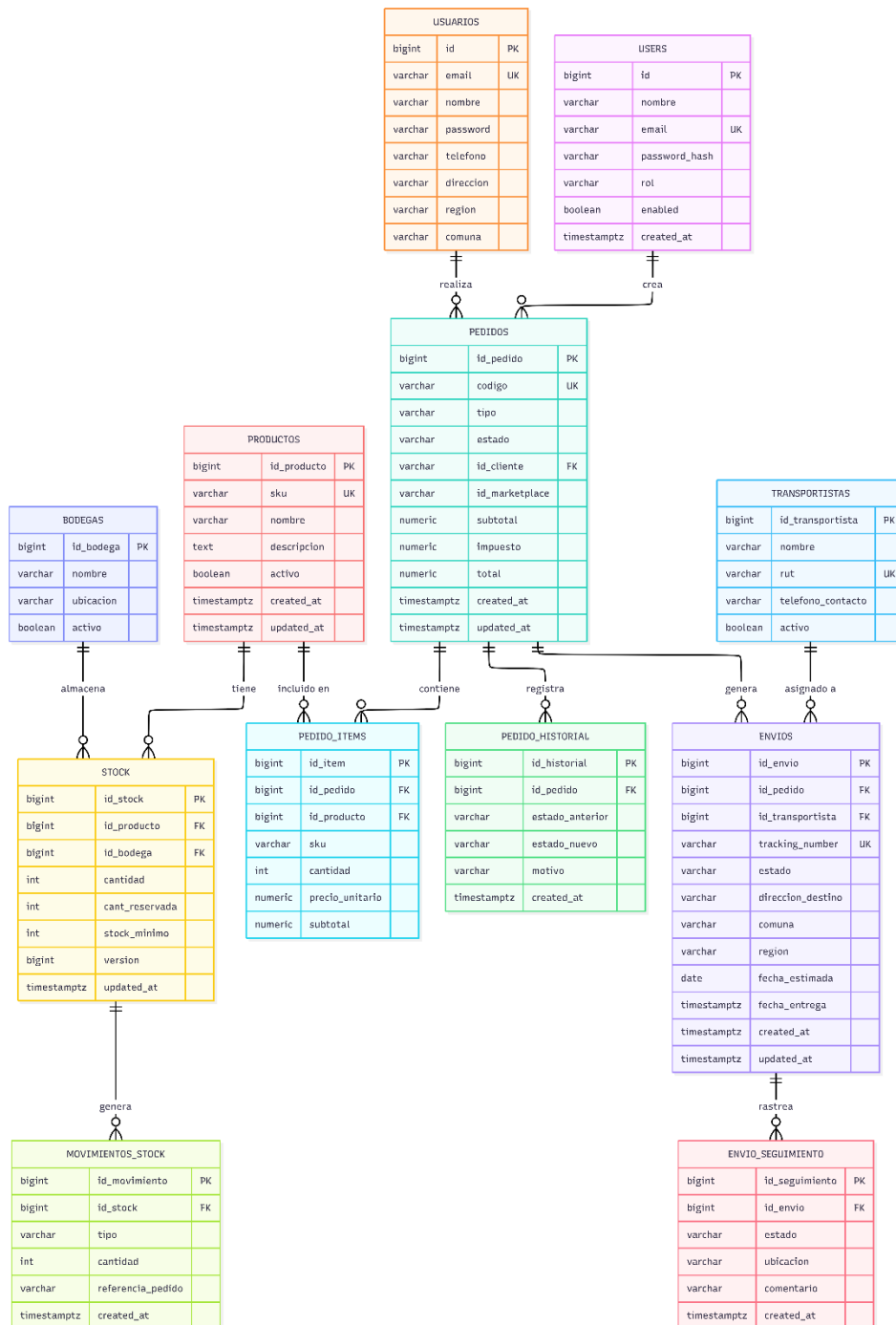
Patrón	Dónde	Problema que resuelve
Repository	Repository (Spring Data JPA) en los 5 MS	Encapsula el acceso a datos; facilita el testeo con mocks
Service Layer	Service + ServiceImpl	Separa el qué del cómo; ubica transacciones y reglas de negocio
DTO (records)	dto/Request,Response	Evita exponer entidades JPA; DTOs inmutables y validables
State Machine	ms-order (7 estados), ms-shipping (5 estados)	Transiciones como datos (Map); transición ilegal → HTTP 409
Optimistic Locking	@Version en Stock (ms-inventory)	Concurrencia en stock sin bloqueos pesimistas; 2.º escritor → 409
Aggregate Root	Order+items+historial, Stock+movimientos, Shipment+seguimiento	Modificaciones a las hijas siempre vía la raíz, misma transacción
Saga / Composite Service	checkout en el BFF	Consistencia eventual entre MS con compensaciones best-effort
Circuit-breaker-lite	clientes HTTP del BFF	AbortController + timeout + degradación parcial del dashboard
RFC 7807 Problem Detail	GlobalExceptionHandler en cada MS y BFF	Errores estandarizados; un solo parser en el frontend
Schema Validation (zod)	BFF	Valida la entrada antes de tocar los MS (defensa en profundidad)

Sobre arquetipos: los 5 microservicios comparten una estructura idéntica (mismos plugins de Gradle, misma disposición de paquetes, mismo Dockerfile), de modo que el build.gradle plantilla cumple el rol de arquetipo reutilizable. La justificación de Gradle frente a Maven está en analisis-patrones-arquetipos.pdf §5. (docs/referencias)

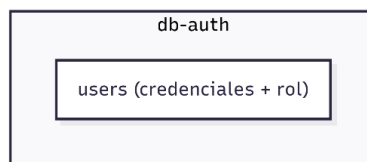
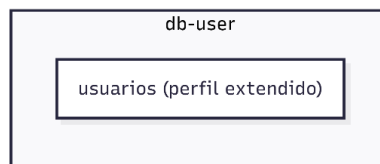
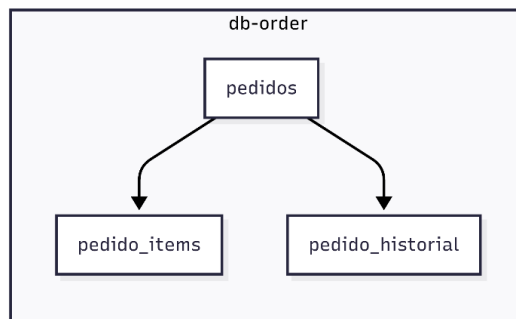
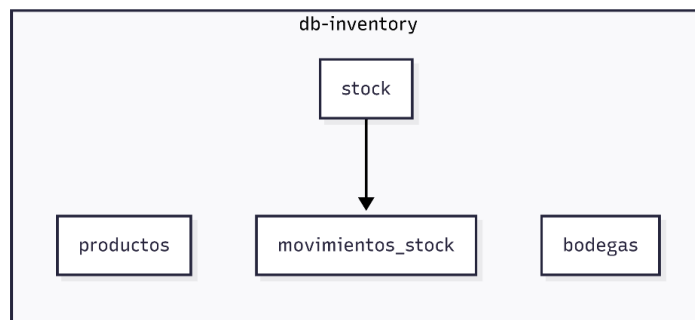
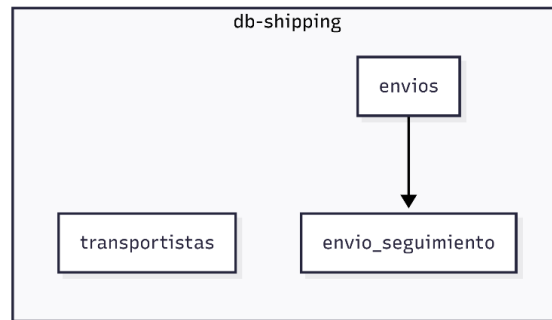
4. Modelo de datos

Cada microservicio es dueño exclusivo de su base (**DB-per-Service**); no existen FK cruzadas. Los enlaces entre dominios son **IDs lógicos** que el BFF resuelve.

Enlaces lógicos entre servicios (no son FK, por DB-per-service):
 pedido_items.id_producto → productos, envios.id_pedido → pedidos,
 movimientos_stock.referencia_pedido → pedidos.codigo.



4.1 Agregados por servicio



4.2 Enlaces lógicos entre dominios (sin FK cruzada)

Origen	Apunta a	Naturaleza
pedido_items.id_producto	inventory.productos	ID lógico (BFF lo resuelve al crear pedido)
pedidos.id_cliente / id_marketplace	Sistema externo (ERP/CRM, marketplaces)	ID lógico externo
movimientos_stock.referencia_pedido	order.pedidos.codigo	ID lógico (al reservar stock)
envios.id_pedido	order.pedidos	ID lógico (envío creado desde pedido aprobado)

4.3 Máquinas de estado



Pedido (ms-order). El envío (ms-shipping) sigue CREADO → ASIGNADO → EN_RUTA → ENTREGADO, con INCIDENCIA como rama lateral reintentable desde EN_RUTA.

4.4 Convención de nomenclatura

La API pública (paths, JSON, scopes) y las entidades Java están en inglés; los nombres físicos de tablas y columnas se mantienen en español por historia (pedidos, id_producto, cantidad...), mapeados con `@Column(name="...")`. La excepción es ms-auth, cuya tabla users está en inglés.

5. Estrategia e implementación de pruebas

5.1 Enfoque

Las pruebas son unitarias con JUnit 5 + Mockito. Se testea la lógica de negocio (servicios, validaciones, máquinas de estado, generación de códigos, manejo de errores) aislando las dependencias con mocks de los repositorios. Por diseño no

requieren base de datos, lo que las hace rápidas y reproducibles en cualquier entorno, incluido el runner de CI.

La cobertura se mide con JaCoCo 0.8.13, que emite reporte XML consumido luego por SonarQube. Se excluyen del análisis los elementos sin lógica (DTOs records, entidades JPA y la clase Application).

5.2 Ejecución de pruebas

Las pruebas fueron ejecutadas mediante Gradle utilizando el siguiente comando:

gradlew clean test jacocoTestReport

5.2.1 Evidencia de ejecución

```
C:\Proyectos\smart-logix\smartlogixs\backend\ms-user>gradlew clean test jacocoTestReport
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

> Task :compileJava
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by lombok.permit.Permit
WARNING: Please consider reporting this to the maintainers of class lombok.permit.Permit
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release

> Task :test
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8

OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath
has been appended

BUILD SUCCESSFUL in 17s
6 actionable tasks: 6 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.4.1/userguide/configura
tion_cache_enabling.html
C:\Proyectos\smart-logix\smartlogixs\backend\ms-user>
```

5.2.2 Evidencia de cobertura

ms-usuario

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.smartlogix.user		0 %		n/a	2	2	3	3	2	2	1	1
cl.smartlogix.user.config		79 %		n/a	2	6	1	15	2	6	0	2
cl.smartlogix.user.controller		80 %		78 %	5	20	9	45	2	13	0	2
cl.smartlogix.user.dto		100 %		n/a	0	4	0	11	0	4	0	3
cl.smartlogix.user.service		100 %		91 %	1	15	0	36	0	9	0	1
Total	58 of 483	87 %	4 of 26	84 %	10	47	13	110	6	34	1	9

inventario

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.smartlogix.inventory.model		63 %		n/a	4	6	10	16	4	6	2	4
cl.smartlogix.inventory.service		97 %		68 %	5	36	2	86	0	28	0	3
cl.smartlogix.inventory		0 %		n/a	2	2	3	3	2	2	1	1
cl.smartlogix.inventory.dto		100 %		n/a	0	12	0	40	0	12	0	8
cl.smartlogix.inventory.controller		100 %		n/a	0	23	0	42	0	23	0	4
cl.smartlogix.inventory.config		100 %		n/a	0	2	0	10	0	2	0	1
Total	39 of 1.004	96 %	5 of 16	68 %	11	81	15	197	6	73	3	21

pedido

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.smartlogix_order.model		82 %		n/a	3	7	8	25	3	7	1	4
cl.smartlogix_order		0 %		n/a	2	2	3	3	2	2	1	1
cl.smartlogix_order.service		99 %		80 %	2	17	0	66	0	12	0	1
cl.smartlogix_order.dto		100 %		n/a	0	7	0	25	0	7	0	5
cl.smartlogix_order.controller		100 %		100 %	0	12	0	28	0	11	0	2
cl.smartlogix_order.config		100 %		n/a	0	2	0	10	0	2	0	1
Total	26 of 694	96 %	2 of 12	83 %	7	47	11	157	5	41	2	14

ms-envio

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.smartlogix_shipping.controller		56 %		100 %	5	18	18	33	5	16	1	3
cl.smartlogix_shipping.service		84 %		75 %	9	27	7	79	6	21	0	2
cl.smartlogix_shipping.model		71 %		n/a	3	5	8	17	3	5	1	3
cl.smartlogix_shipping		0 %		n/a	2	2	3	3	2	2	1	1
cl.smartlogix_shipping.dto		100 %		100 %	0	12	0	34	0	10	0	7
cl.smartlogix_shipping.config		100 %		n/a	0	2	0	10	0	2	0	1
Total	155 of 832	81 %	3 of 20	85 %	19	66	36	176	16	56	3	17

5.2 Resumen de pruebas por microservicio

Microservicio	N.º de tests	Resultado
ms-auth	21	OK
ms-inventory	54	OK
ms-order	27	OK
ms-shipping	30	OK
ms-user	37	OK
Total	169	0 fallos

6. Calidad y cobertura: análisis con SonarQube

Se levantó una instancia self-hosted de SonarQube Community (vía Docker Compose, con PostgreSQL de respaldo) y se analizó cada microservicio con el plugin org.sonarqube de Gradle, que publica las fuentes, el bytecode y el reporte de cobertura JaCoCo. La definición del stack está en infra/sonarqube/.

Informe Final — Integración, Pruebas y Calidad

☆ SmartLogix · ms-auth Public ✓ Passed Last analysis: 2 hours ago · 451 Lines of Code · Java A 0 Security A 6 Reliability A 11 Maintainability E 0.0% Hotspots Reviewed 70.9% Coverage 0.0% Duplications
☆ SmartLogix · ms-inventory Public ✓ Passed Last analysis: 2 hours ago · 810 Lines of Code · Java A 0 Security A 22 Reliability A 26 Maintainability A — Hotspots Reviewed 90.6% Coverage 0.0% Duplications
☆ SmartLogix · ms-order Public ✓ Passed Last analysis: 2 hours ago · 540 Lines of Code · Java A 0 Security A 11 Reliability A 16 Maintainability A — Hotspots Reviewed 92.3% Coverage 0.0% Duplications
☆ SmartLogix · ms-shipping Public ✓ Passed Last analysis: 2 hours ago · 667 Lines of Code · Java A 0 Security A 11 Reliability A 16 Maintainability A — Hotspots Reviewed 80.1% Coverage 0.0% Duplications
☆ SmartLogix · ms-user Public ✓ Passed Last analysis: 3 hours ago · 354 Lines of Code · Java A 0 Security A 5 Reliability A 13 Maintainability E 0.0% Hotspots Reviewed 87.5% Coverage 0.0% Duplications

5 of 5 shown

6.1 Resultados

Proyecto	Cobertura	Línea	Rama	LOC	Bugs	Vulnerab.	Code Smells
ms-auth	70.9 %	69.5 %	100 %	451	0	0	11
ms-inventory	90.6 %	92.4 %	68.8 %	810	0	0	26
ms-order	92.3 %	93.0 %	83.3 %	540	0	0	16
ms-shipping	80.1 %	79.5 %	85.0 %	667	0	0	16
ms-user	87.5 %	88.2 %	84.6 %	354	0	0	13

Los cinco microservicios superan el umbral del 60 % exigido por la rúbrica, con 0 bugs y 0 vulnerabilidades detectadas. Los code smells restantes son sugerencias de mantenibilidad de baja severidad.

6.2 Cómo reproducir el análisis

1. Subir vm.max_map_count (Docker Desktop / WSL2)

```
wsl -d docker-desktop sysctl -w vm.max_map_count=524288
```

2. Levantar SonarQube

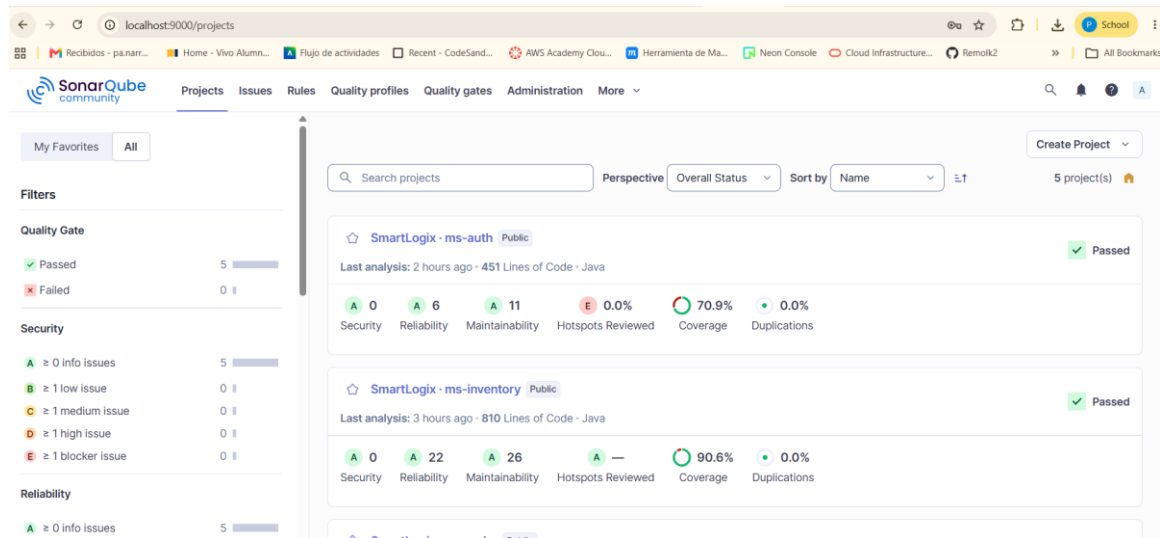
```
docker compose -f infra/sonarqube/docker-compose.yml up -d #  
http://localhost:9000
```

3. Analizar un microservicio (token generado en el dashboard)

```
cd backend/ms-user
```

```
./gradlew test jacocoTestReport sonar -Dsonar.host.url=http://localhost:9000 -  
Dsonar.token=TU_TOKEN
```

El detalle está documentado en [infra/sonarqube/README.md](#).



7. Integración Continua (CI/CD)

Se implementó un pipeline de GitHub Actions (.github/workflows/ci.yml) que se ejecuta en cada push y pull request hacia main y develop, garantizando que las pruebas corran automáticamente en cada cambio.

7.1 Jobs del pipeline

- **Microservicios** (matriz ×5): cada MS compila, corre las pruebas y genera el reporte de cobertura JaCoCo, publicado como artefacto (JDK 25 + Gradle).
- **BFF**: npm ci + chequeo de tipos (tsc --noEmit).
- **Frontend**: npm ci + lint + build de producción (Vite).
- **API Gateway**: validación de krakend.json con krakend check.

7.2 Estado

El pipeline está verde (los 8 jobs pasan). Durante su puesta a punto se corrigieron dos problemas de portabilidad: el bit de ejecución de los gradlew (creados en Windows) y el versionado del lockfile del BFF, necesario para npm ci.

Alcance. El pipeline cubre la Integración Continua (build + test + validación). El despliegue sigue siendo manual (docker compose up o kubectl apply -k), dado que los runners en la nube no tienen acceso al entorno local.

8. Refactorización y mejoras

Durante la etapa de integración se aplicaron mejoras transversales:

- **Migración de la API y el código a inglés** — paths de URL, campos JSON (vía @JsonProperty), variables, métodos y paquetes Java, manteniendo los nombres físicos de la base en español por compatibilidad.
- **Datos semilla** (seed data) para poblar los servicios en entornos de prueba.
- **Limpieza de estructura** del repositorio y actualización de todos los READMEs por componente; la documentación de referencia se consolidó en docs/referencias/.
- **Despliegue en Kubernetes** — manifiestos con kustomize; ingress mediante Traefik (IngressRoute + Middleware: secure-headers, rate-limit, cors-api) que replica las políticas del stack Docker Compose; el ConfigMap de krakend.json se genera a nivel del gateway para no violar el load-restrictor de kustomize (kubectl apply -k infra/k8s funciona sin flags).
- **Calidad continua** — JaCoCo en los 5 MS, análisis con SonarQube y pipeline de CI.
- **Seguridad** — emisión y validación de JWT RS256 asimétrico con ms-auth como emisor y JWKS público, y scopes por endpoint en el gateway.

9. Conclusiones

La etapa EFT consolida SmartLogix como un sistema integrado, probado y verificable:

- 169 pruebas unitarias (0 fallos) cubren la lógica de negocio de los cinco microservicios.
- Cobertura entre 70.9 % y 92.3 % según SonarQube — todos por sobre el 60 % exigido — con 0 bugs y 0 vulnerabilidades.
- Un pipeline de CI ejecuta build, pruebas y validación automáticamente en cada cambio, evitando regresiones.
- La arquitectura final (microservicios desacoplados, DB-per-Service, BFF, gateway + ingress) se mantiene fiel al diseño de EV1, con el ingress Traefik unificado en Docker Compose y kubernetes, y la nomenclatura unificada en inglés a nivel de API y código.

Las mejoras pendientes son menores y están documentadas en el README raíz (p. ej. wildcards multi-segmento en KrakenD y el orden de inicialización de Flyway en Spring Boot 4 para algunos servicios), sin impacto en la operación del sistema.